

Production-Grade Distributed Task Processing Platform

Technologies used & Details:

Project Type: DevOps + Distributed Systems + Cloud-Native Backend

Architecture: Microservices / Event-Driven

Primary Language: Go

containerization: Docker

Orchestration: Kubernetes

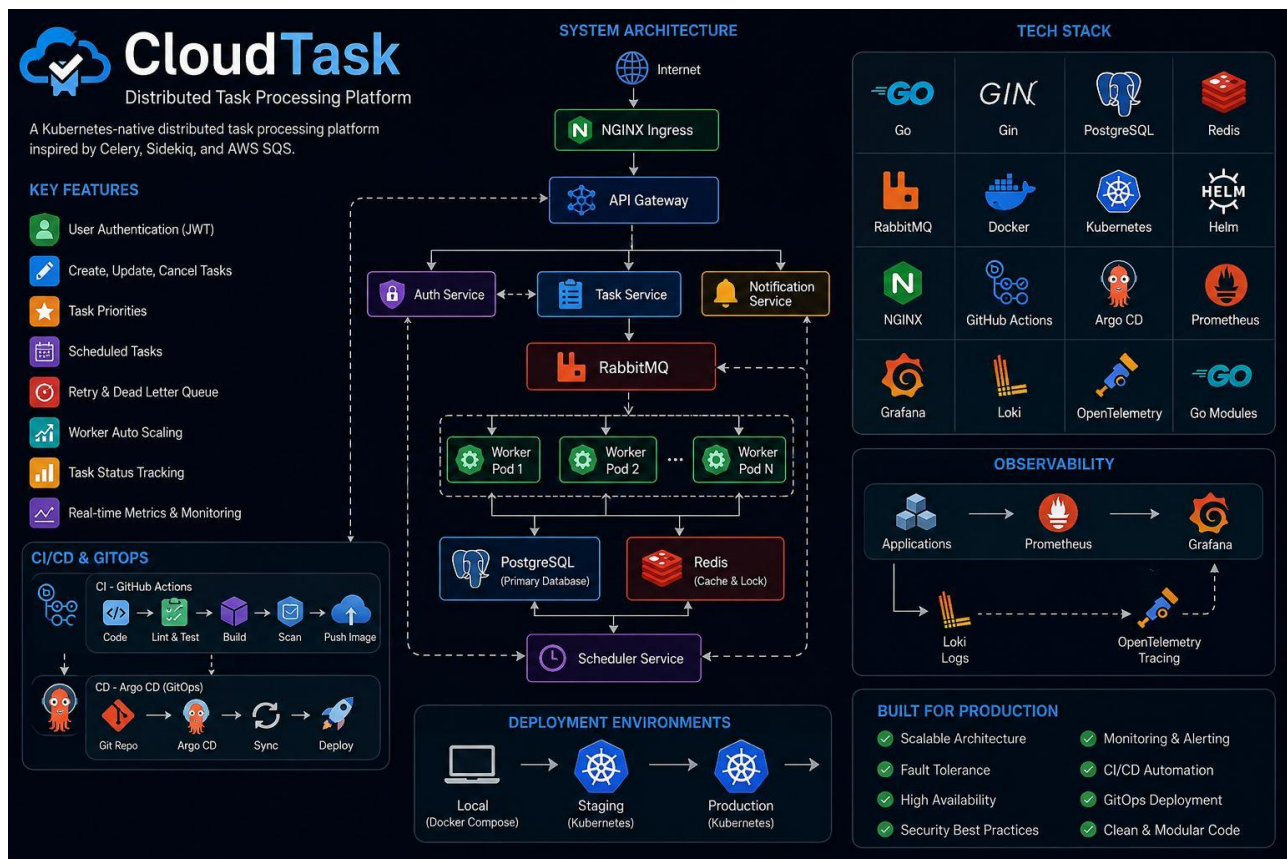
CI/CD: GitHub Actions + Argo CD

Observability: Prometheus + Grafana + Loki + OpenTelemetry

Database: PostgreSQL

Cache: Redis

Message Broker: RabbitMQ



1. Project Overview

- ✓ CloudTask is a Kubernetes native distributed task processing platform inspired by systems such as Celery, Sidekiq, and AWS SQS.
- ✓ The platform allows users to create, schedule, monitor, retry, cancel, and process asynchronous tasks using a distributed pool of worker nodes.
- ✓ The project is designed as a production-oriented engineering system rather than a simple CRUD application. It demonstrates:

- Backend engineering
- Distributed systems
- Asynchronous processing
- Message queues
- Fault tolerance
- Horizontal scaling
- Containerization
- Kubernetes orchestration
- CI/CD
- GitOps
- Infrastructure management
- Observability

- Security
 - Automated testing
- ✓ The primary objective is to demonstrate the ability to design, build, deploy, monitor, and operate a distributed cloud-native system.

2. Problem Statement

- ✓ Many applications need to perform operations that should not block the main API request.
- ✓ Examples include:
- Sending emails
 - Processing files
 - Generating reports
 - Running background jobs
 - Data processing
 - Notifications
 - Scheduled jobs
 - Long-running computations
- ✓ Executing these operations synchronously can result in:
- Slow API responses
 - Request timeouts
 - Poor scalability
 - Increased resource consumption
 - Difficult failure recovery
 - CloudTask solves this problem by separating task submission from task execution.
 - The API accepts a task and places it into a durable message queue. Worker processes consume tasks asynchronously and report their execution status.

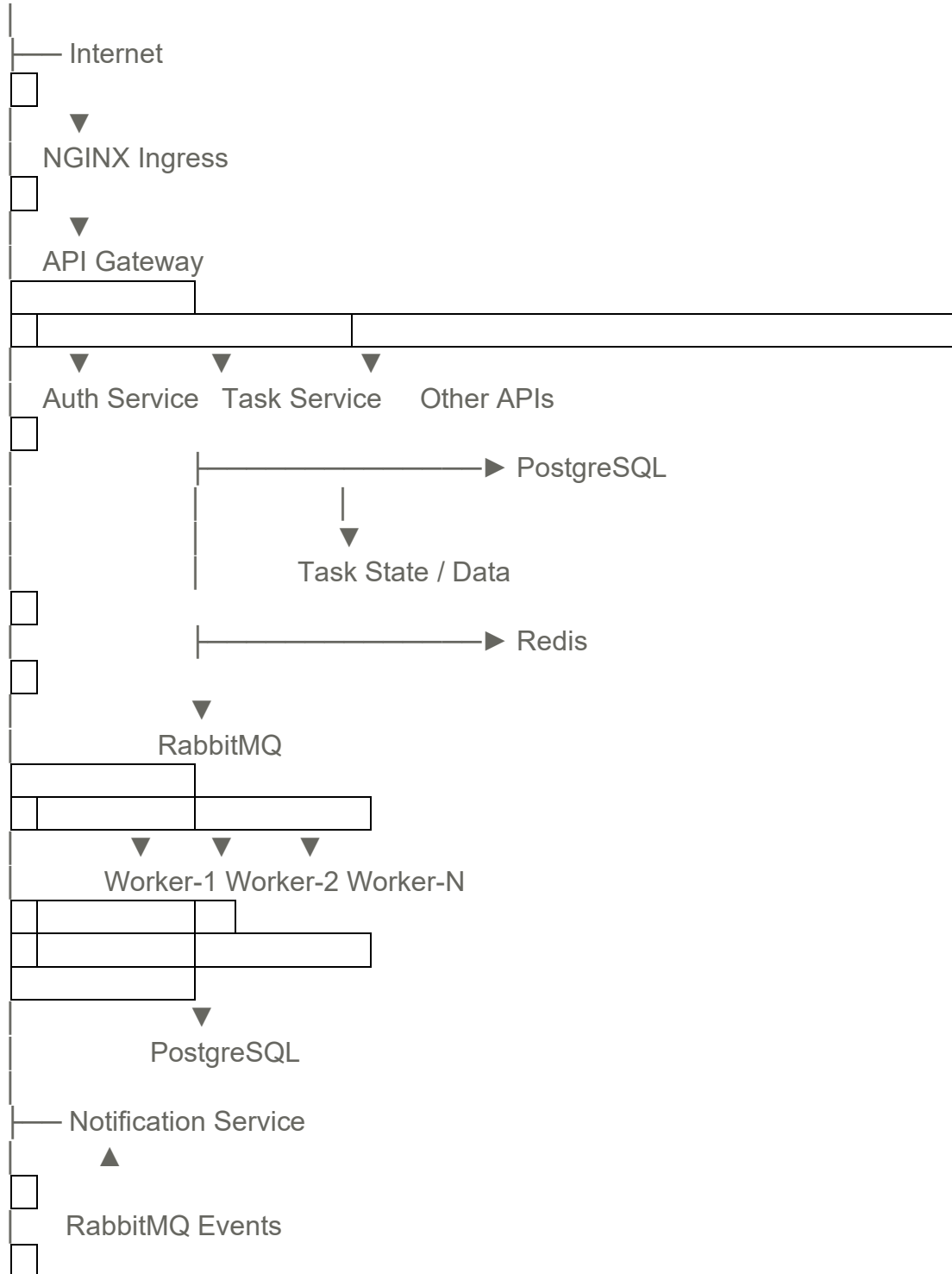
3. Project Objectives:

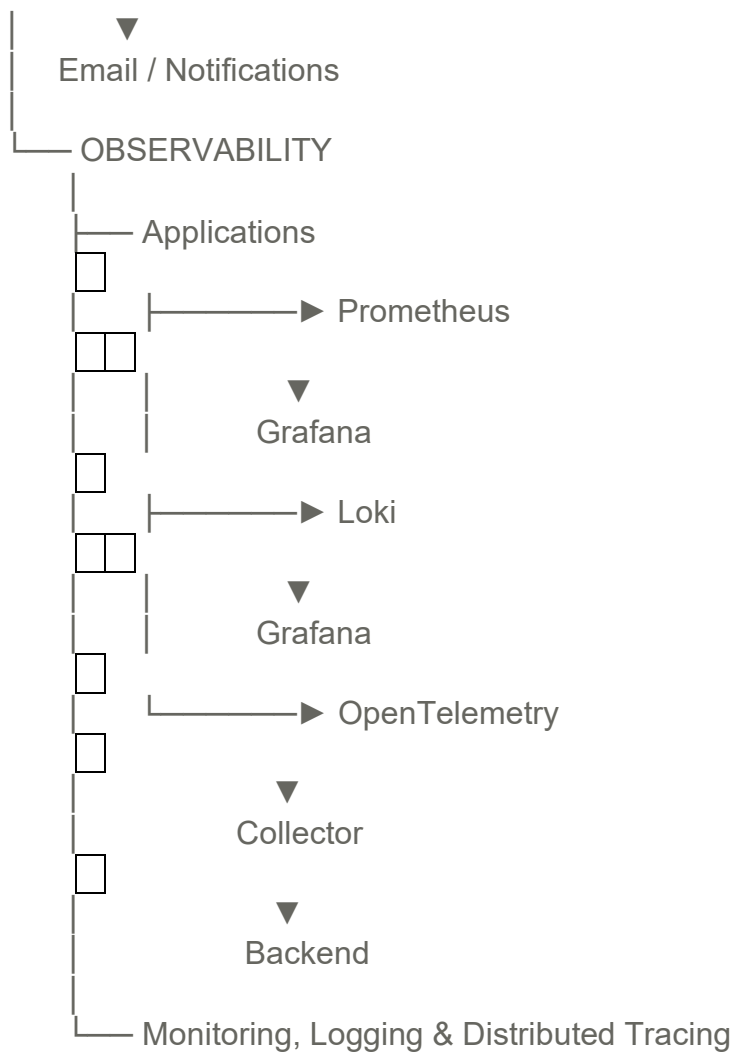
- ✓ CloudTask aims to demonstrate the following engineering capabilities:
1. Design a distributed system.
 2. Build production-quality Go microservices.
 3. Implement asynchronous task processing.
 4. Use RabbitMQ for reliable message delivery.
 5. Implement task retries and dead-letter queues.
 6. Design idempotent task execution.
 7. Deploy services using Kubernetes.
 8. Package deployments using Helm.
 9. Implement GitHub Actions CI.
 10. Implement GitOps-based CD using Argo CD.
 11. Implement centralized logging.
 12. Implement application and infrastructure monitoring.

13. Implement horizontal worker scaling.
14. Apply Kubernetes security controls.
15. Document architecture and operational procedures.

4. High-Level Architecture

CLOUDTASK – SYSTEM ARCHITECTURE





5. Core Services

5.1 API Gateway

The API Gateway acts as the public entry point into CloudTask.

Responsibilities:

- Repeat routing
- Authentication middleware
- Request Validation
- Rate limiting
- Request logging
- Correlation IDs
- API versioning

- Service discovery
- Response handling

The gateway should not contain business logic.

1. Authentication Service

Responsible for user identify and authentication.

Responsibilities:

- User Registration
- User Login
- Password hashing
- JWT Generation
- Token Validation
- User identity management
- Authentication-related database operations

JWTs will be used for stateless authentication.

Passwords must never be stored in plaintext.

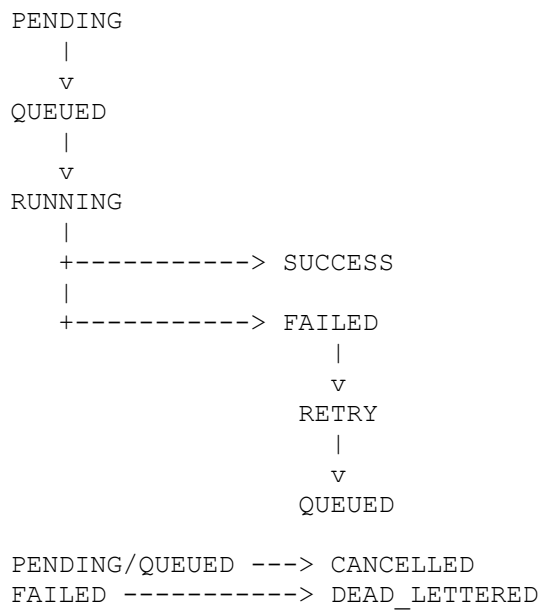
5.2 Task Service

The Task Service manages task lifecycle and task metadata.

Responsibilities

- Create tasks
- Retrieve tasks
- Update tasks
- Cancel tasks
- Retry failed tasks
- Track task status
- Assign priorities
- Manage scheduling metadata
- Persist task state

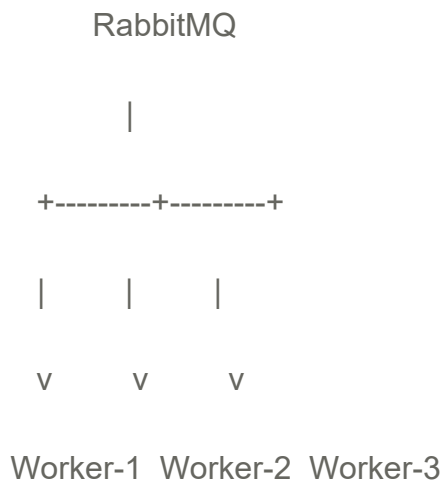
Example Task States:



6. Core Services

Workers are responsible for executing asynchronous tasks.

Multiple worker replicas can run simultaneously.



Worker Responsibilities:

- Consume messages
- Validate tasks
- Execute tasks
- Acknowledge successful messages

- Retry failed tasks
- Handle task timeouts
- Update task status
- Record execution metrics
- Handle graceful shutdown

Workers must be designed to be horizontally scalable.

7. Scheduler Service

The Scheduler handles tasks that should execute at a future time.

Examples:

- Run task at 10:00
- Run task every hour
- Run task every Monday

Responsibilities:

- Detect scheduled tasks
- Determine whether a task is ready
- Publish ready tasks to RabbitMQ
- Prevent duplicate scheduling
- Update scheduling state

The scheduler must be designed carefully to avoid duplicate task publication when multiple scheduler replicas are running.

8. Notification Service

The Notification Service handles asynchronous notifications.

Potential notification types:

- Email
- Task completion notification
- Task failure notification
- System alerts

The service should consume notification events rather than tightly coupling notifications to the Task Service.

9. PostgreSQL

PostgreSQL is the system of record.

Potential tables:

- users
- tasks
- task_attempts
- task_schedules
- idempotency_keys
- notifications

PostgreSQL stores durable business state. It should not be treated as a replacement for RabbitMQ.

10. Reddish

Redis is used for high-speed temporary or frequently accessed data.

Potential use cases:

- Caching
- Rate limiting
- Distributed locks
- Idempotency
- Short-lived task metadata
- Scheduler coordination

Redis should not be treated as the primary source of truth for durable task state.

11. RabbitMQ

RabbitMQ is the asynchronous messaging backbone of CloudTask.

It decouples task producers from task consumers.

Example:

Task Service

|

v

RabbitMQ Exchange

|

+-----+

|

|

v

v

Priority Queue Other Queue

|

v

Workers

RabbitMQ will also support:

- Acknowledgements
- Retries
- Dead-letter exchanges
- Dead-letter queues
- Priority queues
- Consumer concurrency
- Durable queues
- Persistent messages

12. Distributed Systems Model

CloudTask will use an at-least-once delivery model.

This means a task may be delivered more than once in certain failure scenarios.

For example:

RabbitMQ

|

v

Worker receives task

|

v

Task executes

|

X Worker crashes before ACK

|

v

RabbitMQ redelivers task

Therefore, CloudTask must implement idempotent task execution.

Exactly-once execution will not be claimed.

The system will instead use:

At-least-once delivery + Idempotent processing = Reliable task execution

13.Task Idempotency

Every task should have a unique identifier.

Example:

task_id = 7f7c4b5e-....

Workers must ensure that retrying or redelivering a task does not incorrectly execute its side effects multiple times.

An idempotency mechanism may use:

- PostgreSQL
- Redis
- Unique constraints
- Task execution records

The final implementation will choose the appropriate mechanism based on the task type and consistency requirements.

14.Retry Strategy

Failed tasks should not immediately retry indefinitely.

CloudTask will implement controlled retries.

Example:

Attempt 1

|

X Failure

|

v

Backoff

|

Attempt 2

|

X Failure

|

v

Backoff

|

Attempt 3

|

X Failure

|

v

Dead Letter Queue

An exponential backoff strategy will be used.

Example:

1st retry: 5 seconds

2nd retry: 15 seconds

3rd retry: 45 seconds

4th retry: 135 seconds

The exact values will be configurable.

15. Dead Letter Queue

Tasks that cannot be successfully processed after the configured retry limit will be routed to a Dead Letter Queue.

Task Queue

|

v

Worker

|

X

|

Retry

|

X

|

Retry

|

X

|

Dead Letter Queue

DLQ metrics and alerts will be exposed through the monitoring stack.

16.Task Priority

CloudTask will support task priorities.

Example:

- Priority 10 -> Critical
- Priority 5 -> Normal
- Priority 1 -> Low

Higher-priority tasks should be processed before lower-priority tasks where RabbitMQ queue configuration and worker behavior permit.

Priority behavior will be documented clearly because queue-level priority does not guarantee strict global ordering in a distributed worker system.

17.Kubernetes Architecture

CloudTask will be deployed to Kubernetes.

Core Kubernetes resources:

- Namespace
- Deployments
- StatefulSets
- Services
- Ingress
- ConfigMaps
- Secrets
- PersistentVolumeClaims
- HorizontalPodAutoscalers
- NetworkPolicies
- ServiceAccounts
- Roles

- RoleBindings

18. Kubernetes Workloads

Stateless services:

- API Gateway
- Auth Service
- Task Service
- Notification Service
- Worker

Stateful infrastructure:

- PostgreSQL
- Redis
- RabbitMQ

For a serious production deployment, managed PostgreSQL/Redis/RabbitMQ or Kubernetes operators may eventually be preferred over self-managed StatefulSets.

The initial Kubernetes implementation will nevertheless demonstrate StatefulSets, persistent storage, services, and health checks.

19. Kubernetes Health Checks

Each service should expose appropriate health endpoints.

Example:

- /health/live
- /health/ready

Liveness

Determines whether the container should be restarted.

Readiness

Determines whether the pod should receive traffic.

The distinction is important:

- Liveness = "Should Kubernetes restart me?"
- Readiness = "Can I receive traffic?"

20. Resource Management

Every Kubernetes workload should define:

resources:

requests:

- cpu: ...
- memory: ...

limits:

- cpu: ...
- memory: ...

This prevents uncontrolled resource consumption and provides Kubernetes with the information required for scheduling and autoscaling.

21. Horizontal Pod Autoscaling

Workers will be horizontally scalable.

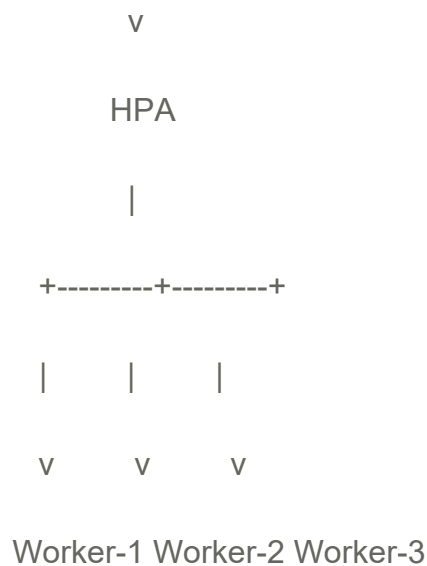
Example:

RabbitMQ Queue

|

Queue depth increases

|



The final autoscaling implementation may use CPU/memory metrics and, preferably, queue-depth/custom metrics.

22. Kubernetes Network Security

NetworkPolicies will implement least-privilege communication.

Example:

Internet

|

Ingress

|

Gateway

|

Task Service

|

RabbitMQ

|

Workers

Services should not automatically be allowed to communicate with every other service.

23. Docker

Each service will have a production-oriented Docker image.

- The images should use:
 - Multi-stage builds
 - Minimal runtime images
 - Non-root users where practical
 - Explicit exposed ports
 - Health checks where appropriate
 - Environment-based configuration

Example:

Builder Image

|

| go build

v

Compiled Binary

|

v

Minimal Runtime Image

24. Docker Compose

Docker Compose will provide the local development environment.

Initial infrastructure:

- PostgreSQL
- Redis
- RabbitMQ

As services are implemented, they will be added to the Compose environment.

Developers should be able to start the local platform using a single command.

25. CI Pipeline

GitHub Actions will perform continuous integration.

Pipeline:

Push / Pull Request

|

v

Formatting Check

|

v

Lint

|

v

Unit Tests

|

v

Integration Tests

|

v

Build

|

v

Container Build

|

v

Security Scan

|

v

Push Image

CI should fail fast when quality gates fail.

26. Security Scanning

Container and dependency security scanning will be integrated into CI.

Potential checks:

- Dependency vulnerabilities
- Container vulnerabilities
- Secret detection

- Static analysis
- Go security checks

Security tools will be selected based on compatibility and maintainability rather than adding unnecessary tools.

27. GitOps with Argo CD

Argo CD will manage Kubernetes deployments.

The desired model:

Developer

|

v

GitHub Repository

|

v

CI builds image

|

v

Container Registry

|

v

GitOps manifests / Helm values

|

v

Argo CD

|

v

Kubernetes Cluster

The Git repository becomes the source of truth for deployment configuration.

28. Deployment Strategy

Deployments should use Kubernetes rolling updates.

The platform should define:

- Readiness probes
- Liveness probes
- Startup probes where appropriate
- Rolling update strategy
- Pod disruption considerations
- Resource requests
- Resource limits

Production hardening will later address rollback and release safety in greater depth.

29. Monitoring

Prometheus will collect metrics.

Important CloudTask metrics include:

API

- HTTP request count
- HTTP request latency
- HTTP error rate
- Active requests

RabbitMQ

- Queue depth
- Message rate
- Consumer count
- Acknowledgement rate
- Rejected messages
- Dead-lettered messages

Workers

- Tasks processed
- Tasks succeeded
- Tasks failed
- Tasks retried
- Task execution latency
- Active workers

Scheduler:

- Scheduled tasks
- Published tasks
- Scheduler failures
- Scheduling latency

30. Grafana

Grafana dashboards will provide operational visibility.

Planned dashboards:

- CloudTask Overview
- API Performance
- Worker Performance
- RabbitMQ
- Task Processing
- Kubernetes Resources

The primary dashboard should allow an operator to quickly answer:

- Is the system healthy?
- Are requests failing?
- Are queues growing?
- Are workers processing tasks?
- Are retries increasing?
- Are tasks entering the DLQ?
- Are Kubernetes resources saturated?

31. Centralized Logging

Loki will provide centralized log aggregation.

Services should produce structured logs.

Example:

```
{ "level": "info",

  "service": "worker",

  "task_id": "7f7c4b5e",

  "event": "task_completed",

  "duration_ms": 342,

  "timestamp": "..."} }
```

Logs should contain correlation information where possible.

32. Distributed Tracing

OpenTelemetry will be introduced later.

The tracing architecture will eventually resemble:

API Gateway

|

v

Task Service

|

v

RabbitMQ

|

v

Worker

|

v

PostgreSQL

This will allow request and task execution paths to be correlated across services.

33. Repository Structure

cloudtask/

|

└─ services/

| └─ api-gateway/

| └─ auth-service/

| └─ task-service/

| └─ scheduler/

```
|   ├── worker/
|
|   └── notification-service/
|
|
|   ├── pkg/
|
|   ├── auth/
|
|   ├── config/
|
|   ├── database/
|
|   ├── errors/
|
|   ├── events/
|
|   ├── httpclient/
|
|   ├── logger/
|
|   ├── messaging/
|
|   ├── metrics/
|
|   ├── models/
|
|   ├── redis/
|
|   └── validation/
|
|
|   ├── database/
|
|   └── migrations/
```

```
|   └─ seeds/
|
|─ deployments/
|   └─ kubernetes/
|   └─ helm/
|
|─ monitoring/
|   └─ prometheus/
|   └─ grafana/
|   └─ loki/
|   └─ otel/
|
|─ scripts/
|
|─ tests/
|   └─ integration/
|   └─ e2e/
|   └─ load/
|
```

```
└─ docs/

├   └─ architecture/

├   └─ adr/

├   └─ api/

├   └─ deployment/

├   └─ operations/

├

└─ .github/

    └─ workflows/

├

└─ docker-compose.yml

└─ Makefile

└─ go.mod

└─ go.sum

└─ .env.example

└─ .gitignore

└─ .dockerignore

└─ README.md
```

34. Development Roadmap

Phase 1 — Infrastructure Foundation

- Initialize repository
- Configure Go project
- Create directory structure
- Configure Docker Compose
- PostgreSQL
- Redis
- RabbitMQ
- Health checks
- Environment configuration
- Makefile
- Initial documentation

Phase 2 — Authentication

- Auth Service
- User registration
- Login
- Password hashing
- JWT generation
- JWT validation
- Authentication tests
- Database migrations
- Docker integration

Phase 3 — Task Service

- Task model
- Task API
- Task CRUD
- Task states
- Priority support
- Task persistence
- RabbitMQ publishing
- Retry API
- Cancel API

Phase 4 — Worker

- RabbitMQ consumer
- Task execution
- Acknowledgements
- Retry handling
- Exponential backoff
- Idempotency
- DLQ
- Graceful shutdown
- Worker metrics

Phase 5 — Scheduler

- Scheduled tasks
- Scheduling persistence
- RabbitMQ publishing
- Duplicate prevention
- Scheduler metrics

Phase 6 — Notification Service

- Event consumption
- Notification processing
- Notification status
- Failure handling

Phase 7 — API Gateway

- Request routing
- JWT middleware
- Rate limiting
- Request logging
- Correlation IDs
- API versioning

Phase 8 — Kubernetes

- Deployments
- Services
- StatefulSets
- ConfigMaps
- Secrets
- PersistentVolumeClaims
- Ingress
- HPA
- NetworkPolicies
- RBAC
- Resource requests/limits

Phase 9 — CI

- Formatting
- Linting
- Unit tests
- Integration tests
- Docker builds
- Security scanning
- Image publishing

Phase 10 — GitOps

- Helm
- Argo CD
- Environment configuration
- Rolling deployments
- Health checks
- Deployment rollback strategy

Phase 11 — Observability

- Prometheus
- Grafana
- Loki
- Structured logging
- Alerting

- OpenTelemetry

Phase 12 — Production Hardening

- Security review
- Dependency management
- Network isolation
- Secret management
- Rate limiting
- Resource tuning
- Failure testing
- Load testing
- Backup strategy
- Disaster recovery documentation
- Operational runbooks

35. Testing Strategy

CloudTask will use multiple testing layers.

Testing Pyramid:

E2E

/ \

Integration

/ \

Unit Tests

Unit Tests

Test:

- Business logic
- Validation

- Retry calculations
- State transitions
- Authentication logic

Integration Tests

Test:

- PostgreSQL
- Redis
- RabbitMQ
- Service interactions
- End-to-End Tests

Test complete workflows:

Register

↓

Login

↓

Create Task

↓

Queue Task

↓

Worker Processes

↓

Task Completed

Failure scenarios should also be tested.

36. Failure Scenarios

CloudTask should explicitly test distributed-system failures.

Examples:

Worker Crash

Worker

|

X Crash

|

v

RabbitMQ

|

v

Message Redelivery

Database Failure

The service should fail gracefully and expose appropriate health/metrics information.

RabbitMQ Failure

Producers and consumers should handle connection failures and reconnect according to controlled policies.

Duplicate Delivery

Workers must safely handle redelivered messages.

Task Failure

Failed tasks should follow the retry policy and eventually enter the DLQ.

37. Security Requirements

CloudTask should follow security best practices.

Requirements include:

- Password hashing
- JWT validation
- Secret management
- HTTPS in production
- Input validation
- Authorization checks
- Least-privilege Kubernetes RBAC
- NetworkPolicies
- Non-root containers
- Dependency scanning
- Container scanning
- No secrets committed to Git
- Secure configuration management
- Rate limiting
- Audit logging

38. Production Hardening

The final phase will focus on operational maturity.

Areas include:

- Resource tuning
- High availability
- Database backups
- Disaster recovery

- Security hardening
- Network isolation
- Failure testing
- Load testing
- Capacity planning
- Alerting
- Runbooks
- Dependency updates
- Release management

39. Architecture Decision Records

Important architectural decisions will be documented under:

docs/adr/

Examples:

- 001-monorepo.md
- 002-rabbitmq.md
- 003-at-least-once-delivery.md
- 004-idempotency.md
- 005-postgresql-source-of-truth.md
- 006-redis-usage.md
- 007-kubernetes.md
- 008-gitops-argo-cd.md

Each ADR should document:

- Context
- Decision
- Alternatives Considered
- Consequences

This makes the repository demonstrate engineering judgment rather than merely technology usage.

40. Git Commit Strategy

Commits should be small, meaningful, and scoped.

Examples:

- chore(repo): initialize CloudTask monorepo
- feat(infra): add PostgreSQL development container
- feat(infra): add Redis development container
- feat(infra): add RabbitMQ development container
- feat(auth): implement user registration
- feat(auth): add JWT authentication
- feat(task): implement task creation API
- feat(worker): add RabbitMQ task consumer
- feat(worker): implement retry handling
- feat(worker): add dead letter queue support
- feat(k8s): deploy CloudTask services
- ci: add Go test and lint pipeline
- ci: add container security scanning
- feat(gitops): add Argo CD application
- feat(observability): add Prometheus metrics
- feat(observability): add Grafana dashboards

41. Definition of Done

A CloudTask feature is considered complete only when applicable items are satisfied:

- ☐ Architecture decision documented
- ☐ Implementation completed
- ☐ Unit tests added
- ☐ Integration tests added where required
- ☐ Error handling implemented
- ☐ Logging implemented
- ☐ Metrics implemented where appropriate
- ☐ Docker configuration updated
- ☐ Kubernetes configuration updated where applicable
- ☐ Helm configuration updated where applicable
- ☐ Documentation updated
- ☐ Security considerations reviewed
- ☐ Meaningful Git commit created

42. Final Project Outcome

The completed CloudTask platform should demonstrate the ability to design and operate a distributed cloud-native system using modern DevOps practices.

The final system will combine:

Go + Microservices + REST APIs + JWT + PostgreSQL + Redis + RabbitMQ + Docker +
Kubernetes + Helm + NGINX Ingress + GitHub Actions + Argo CD + Prometheus + Grafana + Loki
+ OpenTelemetry

More importantly, the project will demonstrate the engineering principles behind these technologies:

Distributed Systems + Reliability + Scalability + Security + Observability + Automation +
Infrastructure as Code + GitOps

CloudTask is intended to serve as a substantial portfolio project demonstrating production-oriented DevOps, backend engineering, distributed systems, Kubernetes, and cloud-native engineering skills.